

```
=====
                        BUBBLE SORT ALGORITHM
                        in C Language
=====
```

```
=====
PROGRAM DESCRIPTION:
```

Bubble Sort is the simplest sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they're in wrong order. The process repeats until the list is sorted. Named because larger elements "bubble" to the top (end) of the list.

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

int main() {
    int arr[100], n, i, j, temp;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Bubble Sort
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
=====
                        PROGRAM EXPLANATION
=====
```

1. ARRAY DECLARATION AND INPUT:
 - Declares array of maximum 100 elements
 - Takes number of elements from user
 - Reads all elements
2. OUTER LOOP (Number of Passes):

```
for (i = 0; i < n - 1; i++)
```

 - Controls number of passes through array
 - Each pass places one element in final position
 - n-1 passes needed to sort n elements
3. INNER LOOP (Comparisons and Swaps):

```
for (j = 0; j < n - i - 1; j++)
- Compares adjacent elements
- "n - i - 1" reduces comparisons each pass
- Largest unsorted elements bubble to the end
```

4. COMPARISON AND SWAP:

```
if (arr[j] > arr[j + 1])
- Compares adjacent elements
- Swaps if first element is greater
- Uses temp variable for safe swapping
```

5. OUTPUT:

```
- Prints sorted array
```

ALGORITHM STEPS

Bubble Sort Process:

Step 1: Start with pass 1 (i = 0)

Step 2: For pass i:

- a) Compare elements at positions j and j+1
- b) If arr[j] > arr[j+1], swap them
- c) Repeat for all adjacent pairs (j = 0 to n-i-2)

Step 3: After each pass, largest unsorted element is in place

Step 4: Repeat Step 2 for n-1 passes

Step 5: Array is sorted

Key Concept:

"Bubbling" - Larger elements move towards end with each comparison

EXAMPLE WALKTHROUGH

Array: [64, 34, 25, 12, 22]

Goal: Sort in ascending order

PASS 1 (i=0):

```
j=0: Compare 64, 34 → Swap → [34, 64, 25, 12, 22]
j=1: Compare 64, 25 → Swap → [34, 25, 64, 12, 22]
j=2: Compare 64, 12 → Swap → [34, 25, 12, 64, 22]
j=3: Compare 64, 22 → Swap → [34, 25, 12, 22, 64]
Result: Largest (64) bubbled to end
```

PASS 2 (i=1):

```
j=0: Compare 34, 25 → Swap → [25, 34, 12, 22, 64]
j=1: Compare 34, 12 → Swap → [25, 12, 34, 22, 64]
j=2: Compare 34, 22 → Swap → [25, 12, 22, 34, 64]
Result: Second largest (34) in correct position
```

PASS 3 (i=2):

```
j=0: Compare 25, 12 → Swap → [12, 25, 22, 34, 64]
j=1: Compare 25, 22 → Swap → [12, 22, 25, 34, 64]
Result: Third largest (25) in correct position
```

PASS 4 (i=3):

```
j=0: Compare 12, 22 → No swap → [12, 22, 25, 34, 64]
Result: Array completely sorted
```

Final Sorted Array: [12, 22, 25, 34, 64]

COMPLEXITY ANALYSIS

Time Complexity:

Best Case: $O(n)$ - Already sorted (with optimization)
Average Case: $O(n^2)$ - Random order
Worst Case: $O(n^2)$ - Reverse sorted

Detailed Analysis:

Best Case (Sorted Array - with optimization):

Comparisons: $n - 1$
Swaps: 0
Total: $O(n)$

Worst Case (Reverse Sorted):

Pass 1: $n - 1$ comparisons
Pass 2: $n - 2$ comparisons
...
Pass $n-1$: 1 comparison
Total: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 = O(n^2)$
Swaps: Also $n(n-1)/2 = O(n^2)$

Average Case:

Comparisons: $n(n-1)/2 \approx O(n^2)$
Swaps: $n(n-1)/4 \approx O(n^2)$

Comparison Table:

Algorithm	Best	Average	Worst	Space
-----	----	-----	-----	-----
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

ADVANTAGES & DISADVANTAGES

Advantages:

- ✓ Simple to understand and implement
- ✓ No extra space required (in-place sorting)
- ✓ Stable sorting (preserves order of equal elements)
- ✓ Good for small datasets (< 50 elements)
- ✓ $O(n)$ best case for nearly sorted arrays
- ✓ Easy to detect if array is sorted (no swaps in pass)

Disadvantages:

- x $O(n^2)$ time complexity (very inefficient for large datasets)
- x Much slower than other algorithms (QuickSort, MergeSort)
- x Not suitable for real-time systems
- x Poor performance on partially sorted data
- x Many comparisons and swaps in average/worst case
- x Rarely used in practice for real applications
- x Educational purposes mainly

KEY CHARACTERISTICS

Stability:

Stable - Elements with equal values maintain relative order

Example: [3a, 1, 3b, 2]

After sorting: [1, 2, 3a, 3b] (3a and 3b maintain order)

In-Place:

Yes - Requires only $O(1)$ extra space (temp variable)

Comparison-Based:

Yes - Uses comparisons between elements

Adaptive:

Partially - $O(n)$ best case, $O(n^2)$ typical cases

Online:

No - Requires seeing entire array before sorting

Natural:

No - Better suited for arrays than linked lists

OPTIMIZATIONS

Optimization 1: Early Exit (Optimized Bubble Sort)

```
for (i = 0; i < n - 1; i++) {
    int swapped = 0;
    for (j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            // Swap
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
            swapped = 1;
        }
    }
    if (!swapped) break; // Array is sorted
}
```

Benefit: $O(n)$ time for already sorted arrays

Optimization 2: Cocktail Shaker Sort (Bidirectional)

- Sort forward, then backward in alternating passes
- Faster than standard bubble sort
- Still $O(n^2)$ worst case

Optimization 3: Comb Sort

- Uses gap sequence instead of comparing adjacent elements
- Better performance than bubble sort
- Closer to QuickSort performance

WHY BUBBLE SORT IS SLOW

Reasons for Poor Performance:

1. EXCESSIVE COMPARISONS:

- $O(n^2)$ comparisons in average case
- Each comparison takes time
- Worst case: 50 billion comparisons for 1 million elements

2. EXCESSIVE SWAPS:
 - Many unnecessary swaps
 - Each swap writes to memory multiple times
 - Memory writes are slower than reads
3. POOR LOCALITY:
 - Random memory access pattern
 - Cache misses reduce CPU efficiency
 - Modern CPUs favor sequential access
4. INEFFICIENT ALGORITHM:
 - Doesn't use divide-and-conquer
 - Doesn't use clever partitioning
 - Simply brute-force comparison

Example Performance (approximate):

- 100 elements: ~5,000 comparisons
- 1,000 elements: ~500,000 comparisons
- 10,000 elements: ~50,000,000 comparisons
- 1,000,000 elements: ~500,000,000,000 comparisons

PRACTICAL APPLICATIONS

Where Bubble Sort is Used:

1. Educational Purposes:
 - Teaching sorting concepts
 - Learning algorithm analysis
 - Understanding time complexity
2. Very Small Datasets:
 - Arrays with < 10 elements
 - Minimal overhead of comparison algorithms
3. Nearly Sorted Data:
 - With optimization flag, $O(n)$ on nearly sorted
 - Good for data arriving in batches
4. When Simplicity Matters:
 - Embedded systems with limited resources
 - Quick prototyping

Where Bubble Sort is NOT Used:

1. Production Systems:
 - Performance is critical
 - Better algorithms always available
 2. Large Datasets:
 - Unacceptably slow
 - QuickSort or MergeSort much better
 3. Real-Time Applications:
 - Cannot meet strict time requirements
 - $O(n^2)$ is too slow
 4. Competitive Programming:
 - Time limits would cause failure
 - Better algorithms required
-

COMPARISON: BUBBLE vs INSERTION vs SELECTION

Bubble Sort vs Insertion Sort:

- ✓ Both simple to implement
- ✓ Both have $O(n^2)$ average case
- ✗ Bubble has $O(n^2)$ swaps worst case
- ✓ Insertion has $O(n)$ swaps worst case
- ✓ Insertion has $O(n)$ best case
- ✗ Bubble $O(n)$ best case only with optimization
- ✗ Bubble generally slower

Bubble Sort vs Selection Sort:

- ✓ Both have $O(n^2)$ time complexity
- ✓ Both in-place and stable (bubble)
- ✗ Selection not stable
- ✓ Selection fewer swaps: $O(n)$
- ✗ Bubble swaps: $O(n^2)$
- ✓ Selection better for expensive swaps

Bubble Sort vs QuickSort:

- ✗ Bubble $O(n^2)$ average case
- ✓ QuickSort $O(n \log n)$ average case
- ✗ Bubble unsuitable for large datasets
- ✓ QuickSort efficient for large datasets
- ✓ QuickSort 100x-1000x faster for $n > 100$

VARIATIONS

Variation 1: Descending Order

```
if (arr[j] < arr[j + 1]) { // Changed > to <
    // Swap
}
```

Variation 2: Cocktail Sort (Bidirectional)

```
// Sort from left to right
for (j = 0; j < n - i - 1; j++) { ... }
// Sort from right to left
for (j = n - i - 2; j >= i; j--) { ... }
```

Variation 3: Modified with Swap Counting

```
int swaps = 0;
for (j = 0; j < n - i - 1; j++) {
    if (arr[j] > arr[j + 1]) {
        // Swap
        swaps++;
    }
}
if (swaps == 0) break; // Early exit
```

SUMMARY

Bubble Sort:

- Simplest sorting algorithm
- $O(n^2)$ average and worst case
- $O(n)$ best case (with optimization)
- In-place and stable

- Not practical for real applications
- Good for educational purposes only

Recommendation:

Use other sorting algorithms for actual applications.

For learning purposes, Bubble Sort teaches fundamental concepts.

=====